

Homework 3

Due: @11:55PM, Monday, November 18th

Name: Shikha Nukala Uniqname: desigirl

1. Submit a pdf of your typed or handwritten homework on Gradescope.
2. Your answers should be neat, clearly marked, and concise. Typed work is recommended, but not required unless otherwise stated. Show all your work where requested, and state any special or non-obvious assumptions you make.
3. You may discuss your solution methods with other students, but the solutions you submit must be your own.
4. **Late Homework Policy:** Submissions turned in by 1:00 am the next day will be accepted but with a 5% penalty. Assignments turned in between 1:00 am and 11:55 pm will get a 30% penalty, and any submissions made after this time will not be accepted.
5. When submitting your answers to Gradescope you need to indicate what page(s) each problem is on to receive credit. The grader may choose not to grade the homework if answer locations are not indicated.
6. After each question (or in some cases question part), we've indicated which lecture number we expect to cover the relevant material. So "(L7)" indicates that we expect to cover the material in lecture 7 (but depending on your lecture, it may have been covered later)
7. **The last question is a group question.**
 - You may do it in a homework group of up to two students including yourself (yes, you can do them by yourself if you wish).
 - If you work in a group of two for these questions, list the name of the student you worked with in your assignment. Further, we suggest that you not split these problems up but rather work on the problem as a group.
 - Turn these group questions in as part of your individual submission.
 - **For these questions (and these questions ONLY) you are allowed to copy/paste solutions from the other student in your homework group.**
 - It is an honor code violation if a student is listed as contributing who did not actually participate in working on that problem.

Problem 1: Short answer questions (10 points) (L12-17)

Select True or False for each of the below statements:

- 1) **True** / **False**: Resolving branches earlier in a pipeline will result in a lower CPI than if resolved in later pipeline stages
- 2) **True** / **False**: A pipelined datapath requires fewer hardware components than a multi-cycle processor, as it reuses datapath elements
- 3) **True** / **False**: The single-cycle datapath described in lecture has a faster clock frequency than the multi-cycle datapath.

Select the best choice for each statement:

- 4) Which has the highest frequency of access? **Disk storage** / **register** / **cache**
- 5) Which has the highest access latency? **Disk storage** / **register** / **cache**
- 6) Which makes use of both temporal and spatial locality? **Disk storage** / **register** / **cache**
- 7) Which is physically located closest to the ALU? **Disk storage** / **register** / **cache**

Problem 2: Intro to Pipelining (15 points) (L13)

For this problem consider the LC2K code segment below. Notice that there are no hazards.

Address				
0	lw	0	1	6
1	lw	0	2	7
2	add	3	4	5
3	sw	0	1	7
4	add	1	2	3
5	halt			
6	.fill	4		
7	.fill	5		

- 1) Fill in the timing graph below with the pipeline stage that each instruction is in at the start of each cycle. Recall that the stages are IF, ID, EX, MEM, WB. The first IF has been filled in for you. **[5]**

Instruction \ Cycle #	0	1	2	3	4	5	6	7	8	9
lw 0 1 6	IF	ID	EX	MEM	WB					
lw 0 2 7		IF	ID	EX	MEM	WB				
add 3 4 5			IF	ID	EX	MEM	WB			
sw 0 1 7				IF	ID	EX	MEM	WB		
add 1 2 3					IF	ID	EX	MEM	WB	
halt						IF	ID	EX	MEM	WB

Let the initial values of the registers be as follows:

Register	Value
0	0
1	2
2	-1
3	4
4	7
5	5
6	-2
7	-4

- 2) For the following 3 parts, fill in the contents of the **pipeline registers** at the point in time between cycle A and B. That is, “Cycle 3-4” means fill in the contents of the pipeline registers **after cycle 3 finishes, and before cycle 4 starts**. If a pipeline register is unused or you don’t care about the value, write an “X”. Note that cycle 3-4 doesn’t follow cycle 0-1, etc. [10]

Example: **Cycle 0-1**

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	lw	Opcode:	noop	Opcode:	noop	Opcode:	noop
PC Plus 1:	1	PC Plus 1:	X	aluResult:	X	writeData:	X
		regA val:	X	branchTarget	X		
		regB val:	X	regB val:	X		
		offset:	X	eq ? True	X		

i. Cycle 3-4

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	sw	Opcode:	add	Opcode:	lw	Opcode:	lw
PC Plus 1:	4	PC Plus 1:	3	aluResult:	7	writeData:	4
		regA val:	4		X		
		regB val:	7	regB val:	X		
		offset:	X		X		

ii. Cycle 4-5

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	add	Opcode:	sw	Opcode:	add	Opcode:	lw
PC Plus 1:	5	PC Plus 1:	4	aluResult:	11	writeData:	5
		regA val:	0		X		
		regB val:	2	regB val:	X		
		offset:	7		X		

iii. Cycle 6-7

IF/ID		ID/EX		EX/MEM		MEM/WB	
Opcode:	add	Opcode:	halt	Opcode:	add	Opcode:	sw
PC Plus 1:	7	PC Plus 1:	6	aluResult:	9	writeData:	X
		regA val:	X		X		
		regB val:	X	regB val:	X		
		offset:	X		X		

Problem 3: Data Hazards (15 points) (L14)

For this problem, reference the following piece of assembly code. All parts of this problem use the 5-stage LC2K pipeline datapath *as presented in lecture*. (2 stalls after lw for dependent instruction) Assume all other memory locations are initialized to zero.

lw	0	1	one	//lw1 - first so normal
lw	0	2	num2	//lw2 - no hazard here
lw	1	3	num1	//lw3 - 1 stall ds 0 for df
add	3	3	3	//add1 - 2 stall ds 1 df
lw	1	3	3	//lw4 - no hazard
add	1	1	2	//add2 - no hazard
nor	3	3	4	//nor1 - 1 stall d+s
halt				//halt - no hazard
one	.fill	1		
num1	.fill	5		
num2	.fill	3		

- Complete the table assuming **detect-and-stall** is used to deal with data hazards. If an instruction stalls in a particular stage, just list out that stage for each stalled cycle. You may not need all the columns. **[7]**

Cycle:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
lw1	IF	ID	EX	MEM	WB												
lw2		IF	ID	EX	MEM	WB											
lw3			IF	ID*	ID	EX	MEM	WB									
add1					IF	ID*	ID*	ID	EX	MEM	WB						
lw4								IF	ID	EX	MEM	WB					
add2									IF	ID	EX	MEM	WB				
nor1										IF	ID*	ID	EX	MEM	WB		
halt												IF	ID	EX	MEM	WB	

2. Complete the table assuming **detect-and-forward** is used to deal with data hazards. You may not need all the columns. **[8]**

Cycle:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
lw1	IF	ID	EX	MEM	WB												
lw2		IF	ID	EX	MEM	WB											
lw3			IF	ID	EX	MEM	WB										
add1				IF	ID*	ID	EX	MEM	WB								
lw4						IF	ID	EX	MEM	WB							
add2							IF	ID	EX	MEM	WB						
nor1								IF	ID	EX	MEM	WB					
halt									IF	ID	EX	MEM	WB				

Problem 4: CPI of Pipelines (15 points) (L15)

Say you have an LC2K program with the following characteristics:

- 35% lw
- 20% sw
- 15% add/nor
- 30% beq
- 25% of register-writing instructions are immediately followed by an instruction dependent on it. You may assume that it is true no matter what instructions are involved. (lw, and, nor)
- 70% of branches are not taken

1. What would be the expected CPI of your program using **detect-and-stall** to resolve data hazards and **detect-and-stall** for control hazards? Assume we are using the 5-stage pipeline described in class and the stalling is handled by the pipeline. Clearly show your work. [7]

Let's assume there are 10000 instructions. Then we have:

**35% lw and 25% of these are immediately followed by dependent instructions
So 2 noops are needed for $.35 \cdot .25 = 0.088$ percent of instructions
For the beq regardless of taken or not we need 3 noops for .3 percent of instructions
For the add/nor we have 15% and 25% are immediately followed by dependent instructions so 2 noops are needed for $.15 \cdot .25 = 0.038$ instructions
Therefore for $0.088 \cdot 10000 = 880$, $10000 \cdot 0.3 = 3000$, $10000 \cdot 0.038 = 388$
 $880 \cdot 2 = 1760$
 $3000 \cdot 3 = 9000$
 $388 \cdot 2 = 776$
 $1760 + 9000 + 776 = 11536$
 $\text{CPI} = 11536 + 10000 + 1 = 21537 / 10000 = 2.154$**

2. What would be the expected CPI of your program using **detect-and-forward** to resolve data hazards and **predict-not-taken** for control hazards? Assume we are using the 5-stage pipeline described in class. Clearly show your work. [8]

**35% lw and 25% of these are immediately followed by dependent instructions
So 1 noop is needed for $.35 \cdot .25 = 0.088$ percent of instructions
For the add/nor we have 15% and 25% are immediately followed by dependent instructions so 1 noop is needed for $.15 \cdot .25 = 0.038$ instructions**

For the beq we need 3 noops for $.3 \cdot .3 = .09$ percent of instructions because .7 of beq are not taken therefore correctly predicted and do not need stalls.

**Lw: $.088 \cdot 10000 = 880 \cdot 1 = 880$
and / nor: $388 \cdot 1 = 388$
Beq: $.09 \cdot 10000 = 900 \cdot 3 = 2700$
 $2700 + 388 + 880 = 3968$
 $\text{CPI} = 3968 + 10000 + 1 = 13969 / 10000 = 1.397$**

Problem 5: Cache Overhead (15 points) (L19)

A new company has proposed a number of different cache layouts for their system, and you've been asked to come in and calculate the overhead for each of the different caches. Their system uses a cache with 4 KB of data storage capable of addressing exactly 16 GB of byte-addressable memory (you should assume addresses are only as large as needed to access this amount). Stores will be handled by write-back and allocate-on-write policies. Please be sure to show the work for your calculations.

1. The first design is a fully associative cache with a block size of 16 bytes, how many bytes of overhead would the cache keep in total (including any necessary tag bits, valid bits, dirty bits, or LRU bits)? [5]

The total number of blocks in this cache is equal to $1024 \times 4 \text{ KB} = 4096$ (cache size) / 16 which is 256 blocks. The block offset is 4 bits because $\log_2(16)$ is 4. 16 GB = 2^{34} so total address bits is 34. Therefore tag bits is 30 bits since $34 - 4 = 30$. Adding the valid and dirty bits at 1 bit each we get 32 bits for overhead per block. We also have LRU bits as $\log_2(256)$ which is 8 bits so we get 40 bits for overhead per block. The total overhead is number of blocks * overhead per block which is $256 * 40 = 10240$ bits and in bytes this is $10240/8 = \mathbf{1280 \text{ bytes}}$.

2. Their next design utilizes a direct mapped cache with 64 different cache lines. How many bytes of overhead would the cache keep in total (including any necessary tag bits, valid bits, dirty bits, or LRU bits)? [5]

Now the number of lines in the cache is 64 so the total number of blocks is now $4096/64$ which is 64 bytes. The new offset is $\log_2(64) = 6$ bits. We also need the index bits per line which is also 6 bits because the cache lines are each 64 bytes long. The tag is then $34 - 6 - 6 = 22$ bits. This makes each overhead for blocks 24 bits. The total overhead is number of lines * overhead which is $64 * 24 = 1536$ bits or $1536 / 8 = \mathbf{192 \text{ bytes}}$.

3. Finally, they've suggested a 2-way set associative cache with 4 different sets. How many bytes of overhead would the cache keep in total (including any necessary tag bits, valid bits, dirty bits, or LRU bits)? [5]

A 2-way set associative cache with 4 sets so we know the blocks per set are 2. To get the block size we do 4096 (cache size) / $4 * 2$ (number of blocks) which is 512 bytes. For the tag we need $\log_2(512)$ which is 9 bits for the block offset. Additionally for index bits there are $\log_2(4) = 2$ for the number of sets. So tag is $34 - 2 - 9 = 23$ bits long. Here we also add 1 LRU bit for a 2-way set so we get a total of 26 bits for the overhead. Therefore we have the total overhead as number of blocks * overhead which is $8 * 26 = 208$ bits or $208/8 = \mathbf{26 \text{ bytes}}$.

Problem 6: Cache Performance (15 points) (L19)

It's 1987 and Apple has just released the Macintosh II featuring a Motorola 68030 processor. The 68030 contains a state of the art **128 byte** data-cache and uses **byte-addressable** memory. Having taken 370, you know there's more to cache design than just size, so you write a C program to help you determine the block size, number of blocks per set, and number of sets in the cache. You have a function `ACCESS(char *)` that loads the memory at the given pointer. Assume all variables other than the `data[]` array are stored in registers and that the cache has been cleared before each call to `miss_rate(...)`. Assume `data[0]` is at the start of the cache block.

```
#define CACHE_SIZE XX
double miss_rate(int stride) {
    char data[CACHE_SIZE * 2]; // note array is 2*cache size

    for (int i = 0; i < stride; i++){
        for (int j = i; j < (CACHE_SIZE * 2); j += stride) {
            ACCESS(data + j)
        }
    }
}
```

1. In order to test your code, you first run it on a **256 B fully-associative cache** with **32 B blocks**. You set the `CACHE_SIZE` macro to 256. Fill in the miss rate for each stride in the following table. Only answers in the table will be graded. Fractions are OK. **[6]**

Stride	Miss rate
1	1/32
2	1/16
4	1/8
8	1/4
16	1/2
32	1
64	1/32
128	1/32

You now run your program on the Motorola 68030 processor with a **128 byte** data-cache. You set the `CACHE_SIZE` macro to 128 and collect the following data

Stride	Miss rate
1	1/16
2	1/8
4	1/4
8	1/2
16	1
32	1
64	1/16

Using the above data answer the following questions and justify your answers.

2. What is the block size in bytes? **[3]**

Since we see that the miss rate becomes 1 when the stride equals the block size, this occurs at a stride of 16. That means the block size is likely 16 bytes because a new block must be loaded for every access attempt at this stride.

3. What is the associativity of the cache? **[3]**

The cache returns back to the original miss rate of 1/16 at one stride as it does at a stride of 64. Therefore in same set there can be multiple blocks. We know the cache size is 128 bytes and each block is 16 bytes long, so there are 8 blocks in the cache. If the miss rate is 1/16 and it resets at 64 we can do $64/16 = 4$. So this is a 4-way associative cache. A set associative cache.

4. How many sets are there in the cache? **[3]**

Total sets = blocks / associativity = $8 / 4 = 2$ sets.

Problem 7: Looking at a real compiler with a slightly different ISA (Group, 15 points) (L6)

Go to <https://godbolt.org/>. Select the ARM GCC 13.2.0 compiler, the C programming language, and set the compiler options to “-O1” (turning on the optimizer). Note, this is a 32-bit version of ARM and among other differences, the registers are listed with an r rather than an X (so r1 rather than X1). Enter the following code and then answer questions in parts a through d. If you’ve set things correctly, the website bar should look something like this:



```
#include<stdio.h>
int fib(int n)
{
    int a,b;
    if (n <= 1)
        return n;
    a=fib(n-2);
    b=fib(n-1);
    return(a+b);
}

int main ()
{
    int n = 7;
    printf("%d",fib(n));
    return 0;
}
```

You might find

<https://developer.arm.com/documentation/ddi0597/2023-09/Base-Instructions?lang=en> useful as a reference. Also, be sure to notice just how useful the colorization is.

- 1) Copy the assembly code for the function “fib”. Comment each line of assembly for that function explaining what it does. You are going to have to look up some of the instructions online. We are not looking for things that just restate what the assembly instruction does (such as “stores register 2” or “copies register 3 to memory”) but instead explains it in context (“puts the argument onto the stack” or “calls the function fib”). [7]

```
fib:
    push    {r3, r4, r5, lr}
    mov     r4, r0
    cmp     r0, #1
    ble     .L1
    subs    r0, r4, #2
```

```

    bl      fib
    mov     r5, r0
    subs    r0, r4, #1
    bl      fib
    add     r0, r0, r5
.L1:
    pop     {r3, r4, r5, pc}

```

push: The first line saves the values that are in r3,r4,r5 and the link register which contains the return address of the function onto the stack so that they can be accessed even when those registers are overwritten.

mov: The second line moves the value of n which is currently in r0 to r4 because some operations need to be performed on n which is in r0 but we need n for later in r0.

cmp: n which is in r0 is compared to 1 and if it is less than or equal to 1 the branch in the next instruction is taken so this is the base case scenario.

ble: this branch to .L1 label is taken if the previous condition for the base case is true which executes the line after .L1 otherwise it is not taken

subs: if the branch is not taken then the value in r4 is subtracted by 2 and stored in r0 for this instruction. This is the fib(n-2) case in the function for recursion

bl: this branches to the beginning of fib with the new r0 value now so a branch with link will use the current r0 in the new execution of fib and will save that value of r0.

mov: now r0 has f(n-2) result and this value is moved into r5 so it can be edited later without losing r0.

subs: this is the second condition of the fib calculator where the value in r4 is now subtracted by 1 and stored into r4 (the value in r4 is also in r5 so we don't lose that) now we have the n - 1 value

bl: this branches to the beginning of fib with the new r0 value now so a branch with link will use the current r0 in the new execution of fib and will save that value of r0.

add: now r0 has the result of fib(n-1) and r5 has the result of fib(n-2) so now the add adds these two together and stores the result into r0. Now we have the final value of the return

pop: When the base case is reached the values of r3,r4,r5 and pc are popped from the stack and the stack is cleared for any other functions or operations.

- 2) What will be the maximum stack depth of this program (in bytes)? Include main and briefly justify your answer. [2]

From main there are fib calls for 7,6,5,4,3,2,1 for fib(n-1) and fib(n-2) until the base case of 1. For each call to main the push is r3 and lr so two registers are pushed at the beginning of the main function. Each register is of size 4 bytes so main adds $4 \times 2 = 8$ bytes to the stack depth. Next there are 7 fib calls starting from main and each fib call has 4 registers pushed to the stack, each of size 4 bytes. Therefore for all the calls to fib we have $7 \times 4 \times 4 = 112$ bytes to stack depth. Adding these together we have $112 + 8 = 120$ bytes as the maximum stack depth.

- 3) Explain how arguments are being passed, where the return value is being placed, and how caller/callee save issues are being resolved. [2]

The argument being passed is n which is pushed to register $r4$. The return value is being placed in $r0$ after adding the two outcomes for $\text{fib}(n-1)$ and $\text{fib}(n-2)$ calls. This uses callee save because at the beginning of the fib function the registers are saved as a copy and revisited when the function call is returned to after implementing another recursive call.

- 4) Write a short C program which computes combinations recursively¹ and give it to the same compiler. Briefly provide the C code, the assembly code, and explain how arguments are passed. [4]

```
#include <stdio.h>
int comb(int n, int k) {
    if (k == 0 || k == n) {
        return 1;
    }
    return comb(n - 1, k) + comb(n - 1, k - 1);
}
```

Assembly:

```
comb:
    cmp     r1, r0
    it      ne
    cmpne   r1, #0
    bne     .L8
    movs    r0, #1
    bx      lr
.L8:
    push    {r4, r5, r6, lr}
    mov     r6, r1
    subs    r4, r0, #1
    mov     r0, r4
    bl      comb
    mov     r5, r0
    subs    r1, r6, #1
    mov     r0, r4
```

¹ The recursive definition to calculate n choose k can be defined as follows (for n, k non-negative integers):

$C(n, k) = 1$ If $k = 0$ or $n = k$;

Else $C(n, k) = C(n-1, k) + C(n-1, k-1)$

```
bl      comb
add     r0, r0, r5
pop     {r4, r5, r6, pc}
```

Here we can see that n is put into r0 register and k is put into r1 register because r0 is subtracted by 1 first as is in the function while r1 is subtracted for the second comb call which is how k is needed for the second call to comb.